Clothing Assistant Chatbot with Retrieval-Augmented Generation

Final Project Report

Team members :-

Manish S

Madhav V

Neeraj Yadav

Sri Harsha Reddy Sagili

Adarsh Pradhan

1. Abstract

This project develops a Clothing Assistant chatbot that combines store policy assistance with personalized fashion recommendations. Leveraging retrieval-augmented generation (RAG) .This system is split into two core functional components: the Store Policy & Customer Help Desk (Policy RAG) and the AI Stylist (Product RAG).The overall query flow begins with a classification step where the Gemini 2.5 flash LLM determines if a user's query, including conversational history, relates to Policy, Return, or Search and shopping. The query is then routed accordingly

Policy RAG (Store Policy & Customer Help Desk)

This component is responsible for accurately answering customer queries regarding store policies (e.g., return conditions, refunds, and the garment recycling program). The policy knowledge base was synthesized using guidelines from major retailers for academic reference. The Policy RAG utilizes the BAAI/bge-base-en-v1.5 model for embedding documents into ChromaDB. Optimization involved segregating policy information based on sub-headings before chunking to improve retrieval. Evaluation was conducted using the RAGAS framework, which measures generation quality (Faithfulness, Answer Relevancy) and retrieval performance (Context Precision, Context Recall).

Product RAG (AI Stylist)

The AI Stylist provides personalized clothing and outfit suggestions based only on the company catalogue, preventing LLM hallucinations. The product catalog dataset is derived from the H&M Personalized Fashion Recommendations Kaggle competition, licensed exclusively for non-commercial purposes and academic research.

The recommendation process is a multi-stage RAG pipeline:

- 1. Context Addition:
- 2. Query Refinement:
- 3. Retrieval and Reranking

2. Introduction

The Clothing Assistant aims to create an advanced conversational AI for retail clothing stores that not only delivers accurate policy-related information but also acts as a personalized stylist. Unlike existing agents that either focus on policy FAQs or style advice, our solution merges both functionalities with RAG-driven retrieval on comprehensive policy documents and product metadata. This chatbot enhances customer experience by understanding queries, maintaining chat history, and applying fashion knowledge to propose relevant outfit combinations tailored to individual customers.

3. Literature Review (Milestone 1)

● Project Objectives and Core Functionality

The overarching goal of the project is to construct a **Clothing Assistant** chatbot. This system is divided into two operational parts:

- **Part A: Store Policy + Customer Help Desk:** This component is designed to answer customer queries regarding **store policies**, such such as the return policy and upcoming discount information. A critical feature is the ability to handle **personalized return/replacement inquiries** regarding a specific, previously purchased item, requiring the agent to check that product's individual policy.
- **Part B: AI Stylist:** This agent aims to suggest outfits based on both a user's immediate query (e.g., 'suggest me an outfit for a graduation party') and **customer context** (preferences, previous purchases, size, etc.). Key non-functional requirements established in this milestone included:
    - **Preventing hallucinations** by suggesting products **only from the company catalogue**.
    - Having inherent **fashion knowledge** regarding aspects like color matching and occasion-appropriate outfits.
    - Initially utilizing a **quiz designed to capture customer preferences** to enhance recommendations.

- Review of Existing Solutions and Identified Gaps

Following were the gaps identified on existing solutions:

Gap Identified	Proposed Solution
Catalogue Restriction (Hallucination)	Existing solutions often suggested products from external websites. The project mandates the use of only the company's internal product catalogue to ensure availability and prevent irrelevant suggestions.
Query-Based Filtering	Agents did not strictly adhere to customer requests (e.g., suggesting unrelated items when asked for a specific product type). The project aimed to implement query-based filtering using product metadata and embeddings to match specific requests.
Lack of Fashion Sense/Context	Agents failed to provide context-aware pairings (e.g., suggesting improper pants for a specific shirt). The project planned to incorporate sets of rules/prompts for the LLM (reranker) to ensure contextually appropriate outfit suggestions.
Poor Memory Management	Agents failed to properly maintain chat history across turns, forgetting context like the occasion or initial product type. The project planned to implement vector-based conversational memory and intended to use conversation summarizing techniques (like ConversationSummaryMemory) to mitigate increased latency from large token counts

To address these, the project planned to implement **query-based filtering**, incorporate **size information** into product embeddings, and use **vector-based conversational memory**.

The literature review examined various existing solutions, including models like NeuroStylist and Conditional Similarity Networks, noting that these systems often utilized multimodal data (image and text) and specialized training techniques, like **triplet loss**, to cluster compatible products.

The project builds upon key research in recommendation systems and retrieval-augmented models.

Our approach uses text-based features, as feature-rich paired data for images is limited. Store policy assistance relies on RAG assessment frameworks like RAGAS for evaluating factuality and answer relevance, ensuring responses are both grounded and context-aware. Existing solutions mainly struggle with catalog restrictions and query-based filtering, which our system addresses.

- Data Sourcing and Initial Model Strategy

Initial plans for data preparation were defined:

- **Policy Sourcing:** An internal policy document would be generated by referring to guidelines from major online clothing brands (like Myntra and Bewakoof) as baseline references, and then customizing them into a **comprehensive internal policy**.
- **Catalog Sourcing:** The product catalog would contain **all product details along with images**. The initial plan was to **embed the catalog** and store it in a **vector database**.
- **Embedding Choice:** Models like **Sentence-BERT** were initially considered for converting policy and FAQ documents into embeddings for storage in a vector database for retrieval.

- Scoring Metrics and Baselines

Milestone 1 established the metrics necessary to evaluate system performance:

- **Part A (Policy RAG) Scoring:** The primary metric selected was the **Retrieval-Augmented Generation Assessment Suite (RAGAS)**, a specialized

framework focusing on the quality of generation and retrieval. RAGAS measures four key components:

- **Faithfulness:** Measures if the generated answer is factually correct relative to the retrieved documents.
- **Answer Relevancy:** Measures how well the answer addresses the user's original query.
- **Context Precision:** Measures the signal-to-noise ratio of the retrieved context.
- **Context Recall:** Measures if all relevant information needed for the answer was successfully retrieved.
- The project set an initial performance target of a **RAGAS score of 0.8 or above**.

- **Part B (AI Stylist) Scoring:** A tentative custom scoring metric was proposed to evaluate product suggestions based on matching attributes. This metric would assign weightage to factors such as color, material, gender, category, and compatibility. The final score would be scaled based on the filters explicitly provided in the user's query.

The project intended to start with a non-optimized baseline model setup and subsequently **define steps for better search**, increase the LLM’s fashion understanding, and compare performance by testing different embedders.

4. Dataset and Methodology (Milestone 2–3)

The AI Fashion Chatbot is fundamentally supported by a **Retrieval-Augmented Generation (RAG)** pipeline, which enables the large language models (LLMs) to access and utilize external, domain-specific knowledge before generating a response. This architecture relies on two distinct and critical knowledge bases: the synthesized Policy Corpus and the rich H&M Product Catalog.

● Data Sourcing and Structure

Two primary datasets underpin the system, each tailored to one of the chatbot's core objectives (Part A: Policy, Part B: AI Stylist):

A. **Policy Corpus:** A comprehensive policy framework was developed through a combination of creation tools (like ChatGPT) and references drawn from established policy documents of leading clothing retailers, including H&M, Marks & Spencer (M&S), and Levi's. This corpus contains policies covering exchange, refund, and return procedures, as well as loyalty programs and garment recycling initiatives.

B. **H&M Personalized Fashion Recommendations Kaggle Dataset:** This dataset provides the complete product catalog. It includes product metadata, transaction history, customer profiles, and product images. The catalog features essential attributes for search and recommendation, such as product_type_name, colour_group_name, detail_desc (rich product description), Eco_Score, and Return_Type. Crucially, the license for this dataset restricts its use to non-commercial purposes and academic research only.

For the Product RAG component, the data is organized across three tables: **Articles.csv** (product catalog), **Customers.csv** (customer metadata, including age, gender, size, and persona), and **Transactions.csv** (purchase history).

- ## Embedding Strategy and Model Selection

The RAG methodology depends on converting complex data (text and metadata) into numerical embeddings, which are stored in vector databases (VDBs) for rapid retrieval via semantic search.

- **Policy RAG Embedding:** The team selected the BAAI/bge-base-en-v1.5 (BGE) model as the preferred choice for Part A (Policy RAG). BGE is specifically optimized for dense retrieval in RAG systems, balancing high accuracy with efficiency for semantic search.
    - **Preprocessing:** The policy documents underwent a specific chunking strategy: the text was first segregated based on sub-headings before being split into chunks. Each chunk was then embedded using the BGE sentence transformer and persisted in ChromaDB. Keywords (generated using Llama-3.1-8b-instant) and policy titles were also added as metadata to the chunks to enhance retrieval quality.
- **AI Stylist (Product RAG) Embedding:** While the immediate implementation utilized BGE models for text embeddings, FashionSigLIP was identified as the preferred future model for the AI Stylist component. FashionSigLIP is a domain-tuned, multimodal architecture optimized for fashion and e-commerce imagery, offering the best balance of fine-grained fashion understanding and text–image alignment. This is critical for future extensions like searching based on user-uploaded images.
    - **Preprocessing:** For the current Product RAG, unwanted columns (like index codes and category codes) were removed from the catalog data, and the remaining product details were combined, embedded using the BGE model, and saved in a Parquet file for use with a VDB (FAISS).

- Context-Aware Query Flow

The core methodology implements a multi-stage flow centered on context building and LLM classification:

- **Query Classification:** An initial LLM determines if the user's input is related to Policy, Return, or Product Search.
- **Context Building:** For product searches (AI Stylist), extensive customer context is assembled, including age, gender, location, and previous purchase details (style, price point, etc.).
- **Mandatory Information Check:** The pipeline checks if mandatory information (like age and gender) is available for the search. If this context is missing, the LLM will explicitly ask the customer a follow-up question before proceeding.
- **Memory Management (Planned):** To handle lengthy conversations and mitigate latency caused by exceeding token limits, the architecture includes plans to use conversation summarizing techniques, such as ConversationSummaryMemory, possibly utilizing Redis for persistent session storage.

The overall methodology aims to ensure that retrieval is not only semantically relevant but also deeply personalized and grounded in the specific product catalog and store policies.

5. Model Development and Hyperparameter Tuning(Milestone 4)

User context (age, gender, purchase history) is incorporated into query refinement for semantically relevant retrieval.

Hyperparameter tuning explored embedding chunk sizes, overlap words, retrieval strategies, and LLM creativity parameters such as temperature and Top-K selections.

Evaluation showed that the BAAI/bge-base-en-v1.5 model with specific chunk and overlap parameters optimized performance.

The AI stylist follows a three-step process: refining user queries, product catalog search via cosine similarity of embeddings, and reranking top results with weighted customer context for personalized recommendations.

Product RAG Scores

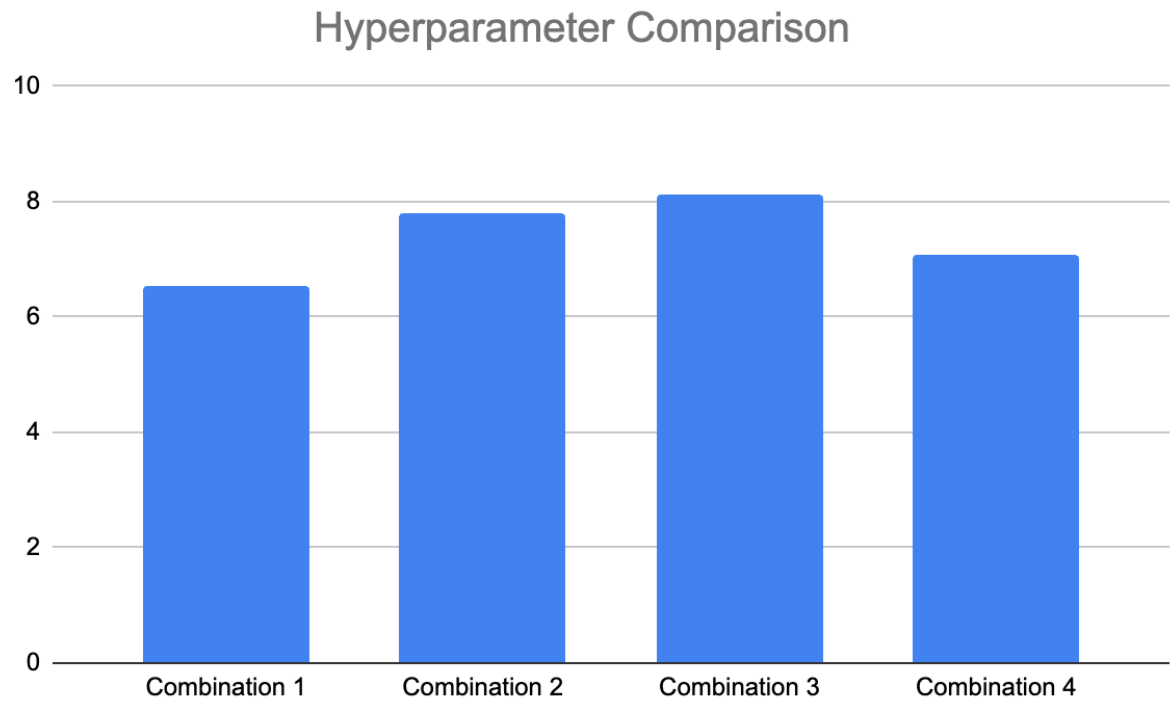
For product recommendations, scores were assessed across ten typical queries and all four hyperparameter combinations, using a ten-point scale. Sample results:

Query	Products	combination 1	combination 2	combination 3	combination 4
1	1	8	9	9	9
	2	4	8	7	6
	3	3	4	4	8
2	1	8	8	8	8
	2	7	9	8	9
	3	8	6	7	7
3	1	8	8	8	9
	2	8	9	9	8
	3	6	7	7	9

	Combination 1		
Query	**Product 1**	**Product 2**	**Product 3**
Q1 – “What about a red one?”	9 (NILS – Red Shirt → right color, ever	8 (SAILOR FRONT PRINT – Red Sweater	5 (CORN CLASSIC TEE – Dar
Q2 – “help me find a blue one”	10 (Retro Slim – Blue Denim → perfec	9 (Skinny 5pkt Trash 1 – Blue Denim → ac	8 (Skinny 5pkt Midnight – Dark
Q3 – “blue jean around 2000 INR”	10 (Skinny 5pkt Midnight – Slim fit and	9 (Skinny 5pkt Premiumprice – Blue → clo	8 (Retro Slim – Blue → fit a bit
Q4 – “red top”	10 (SAILOR FRONT PRINT – Red Sw	6 (Rosie Blouse – Black → cozy fabric but	7 (SAILOR FRONT PRINT – W
Q5 – “shirt that goes with cream pant”	10 (STYLO SLIM SHIRT – White → pr	7 (Sam LS BD – Light Blue → close tone b	4 (Pat LS BD – Black → color
Q6 – “formal shirts for office meetings”	10 (Sam LS BD – Light Blue Shirt → e	6 (SPEED Paprika – Grey Sweater → cas	4 (SPEED Paprika – Black Sw
Q7 – “casual wear like polos or chinos”	10 (Rawley Chinos Slim – Yellowish Br	5 (Reggie Chino Shorts – Black → casual	4 (Slim Straight 5pkt Midway –
Q8 – “stylish yet comfortable for casual outings”	10 (Bernie – Light Blue Top → ideal ma	8 (Tilda Gatherings Top – White Sweater	4 (SPEED Paprika – Black Sw
Q9 – “light summer dress for office & evening”	10 (Topi Dress J – Black → perfect ma	6 (Polly Dress – Light Blue → wrong color,	5 (Sam Dress – Off White → n
Q10 – “trendy tops in pastel colors for brunch”	10 (Bernie – Light Blue Top Garment →	8 (Tulip Jumper – White → close lightness	6 (Tulip Jumper – Grey → neut

Averaged across queries and combinations.

Combination-3 (T=0.5 *T*=0.5, Top-K=12) achieved both high consistency and accuracy as rated by human evaluators.



- Policy RAG development and hyperparameter tuning:

The development initial reliance on fixed chunking strategies, followed by iterative testing of embedding models, which ultimately led to the adoption of a refined, context-based chunking strategy.

Initial Fixed Chunking Strategy and Model Comparison:

- The initial hyperparameter tuning Chunking Strategy Tested:** The initial approach used set chunk sizes and overlaps: * Combination 1: Used BAAI/bge-base-en-v1.5 with Chunk size 700/Overlap 70 (Collection 1) and Chunk size 300/Overlap 30 (Collection 2). Both collections were queried, and chunks with high cosine similarity were chosen. * Combination 2: Used all-MiniLM-L6-v2 with a single collection setup using Chunk size 700/Overlap 70.
- Embedder Model Comparison: Both the BAAI/bge-base-en-v1.5 model and the all-MiniLM-L6-v2 model were tested.
- Performance: The BAAI/bge-base-en-v1.5 model (Combination 1) performed significantly better, achieving an average manual score of 4.8 out of 5 on the test queries, compared to the average score of 2.9 achieved by the all-MiniLM-L6-v2 model (Combination 2).

Transition to Context-Based Chunking Strategy

Despite the superior performance of the BGE model, the testing revealed a fundamental flaw in the fixed chunking methodology:

- **Observation of Failure:** During hyperparameter optimization, the project team found that the initial chunking strategy (where policies were not segregated based on sub-headings) did not work well.
- **Upgrade to Context-Aware Strategy:** Based on these results, it was concluded that the Policy RAG needed an upgrade to become context-aware.

- **Implementation of Context-Based Chunking:** The new strategy mandated that the policy document must be segregated based on sub-headings before chunking. The policy document, which was originally created internally and synthesized from the guidelines of major retailers (H&M, M&S, Levis), was split based on policy sections.

- Final Processing Flow:

The refined chunking strategy involves:

1. Fetching data (e.g., from PDF).
2. Splitting the context based on Policy.
3. Splitting each segregated policy into chunks of size N.
4. Passing each chunk to an LLM (llama-3.1-8b-instant was used in production) to fetch 5–7 keywords for metadata.
5. Embedding the chunks using the BGE Model and adding the keywords and policy title as metadata.
6. Persisting the embeddings in ChromaDB.

This transition significantly improved the system, ensuring that sufficient, relevant policy context was provided within each retrieved chunk, which facilitated more accurate response generation by the LLM. Chunk sizes above 100 words were found to be ideal in the new structure as they provided sufficient context

6. Evaluation & Analysis (Milestone 5)

- Product RAG was evaluated on manually scored test queries focusing on relevance, filter adherence, history usage, and fashion sense.
- Four hyperparameter combinations were tested, where a balanced temperature (0.5) and Top-K (12) setting achieved the highest average score of 8.13/10, balancing creativity and precision.

Hyperparameter Combinations

The following table summarizes the evaluated hyperparameter combinations for product recommendations:

Hyperparameter Combinations	Temperature	Top-K	Refine Prompt	Rerank Prompt
Combination 1	0.2	5	refine_prompt_contextual	rerank_prompt_contextual
Combination 2	0	10	refine_prompt_precise	rerank_prompt_precise
Combination 3	0.5	12	refine_prompt_balanced	rerank_prompt_balanced
Combination 4	0.8	20	refine_prompt_creative	rerank_prompt_creative

Evaluation Metrics (RAGAS)

System output quality was measured with the RAGAS framework, which captures key aspects of retrieval-augmented generation through the following metrics:

Ragas Metric	What it Measures	Significance of a High Score (Closer to 1
Faithfulness	**Fact-Checking (Hallucination)**	The generated answer is truthful and directly supported by the provided context documents.
Answer Relevancy	**User Intent**	The generated answer is fully relevant and directly addresses the user's question, without containing unnecessary or tangential information.
Context Precision	**Noise in Context**	The retrieved context documents are highly focused and contain minimal irrelevant or "noisy" information for answering the question.
Context Recall	**Completeness**	The generated answer incorporates all necessary facts from the context that are needed to provide a comprehensive response to the question.

Policy RAG Scores (RAGAS)

The policy assistant was evaluated with policy-related questions, scoring above 0.9 on faithfulness, answer relevancy, context recall, and context precision across diverse queries:

Question	Faithfulness	Answer Relevancy	Context Precision	Context Recall
How can I return or exchange a product?	0.9	0.9	1	0.9
Which products cannot be returned?	1	1	0.95	1
How long does it take to receive a refund?	0.9	1	1	0.95
Can I exchange a product for another size?	1	1	1	1

Analysis

- Combination 3 achieves the optimal blend of creativity (temperature) and retrieval width (Top-K) for product recommendations, with the highest average scores.
- The RAGAS evaluation confirms both product and policy RAG systems produce responses that are highly faithful, minimizing hallucination and maximizing both context relevancy and recall.
- Qualitative analysis of sample queries further indicates that the system performs best where explicit catalog data matches user intent. Some minor context recall losses occur when necessary procedural steps are only partially represented in the source context.

7. Deployment & Documentation (Milestone 6)

- **Overview:** We chose **Render** as our primary deployment platform because it supports full-stack deployment, provides an easy workflow, and offers a free hosting tier for testing.
- **Frontend Deployment**: The frontend was successfully deployed using Render's static site hosting service.

Live Frontend URL: https://group-5-ds-ai-lab-project.onrender.com/

Status:

- Deployment successful Fully functional Responsive and publicly accessible
- The frontend includes all UI components such as:
  - Product search interface
  - Dashboard
- The deployment works smoothly because the frontend has minimal memory and compute requirements.

- **Backend Deployment Attempt:** We attempted to deploy our backend (API server) on Render using a separate service.

Backend Attempt URL: https://group-5-ds-ai-lab-project-1.onrender.com/

Status:

- Deployment failed (service not running)
- **Failure Reason:**
  - Large machine learning model
  - Embedding vectors
  - Product dataset
  - Additional mapping and preprocessing files All these together require **high memory**, which exceeds Render's free tier limits.
  - **Typical free-tier limits: 512 MB**
  - **But our backend requires: More than 2–4 GB RAM** (due to model + data + embeddings)

- **As a result:**
  - Render killed the backend container during startup
  - Out-of-memory (OOM) errors occurred
  - The server could not boot successfully
  - **So the backend URL exists, but the service does not run due to insufficient memory.**

- **Solutions for Successful Backend Deployment**

        - **Option A: Upgrade Render Plan (Paid)**
            - Render's paid tiers support higher memory (2 GB, 4 GB, 8 GB etc.). Upgrading would allow:
            - Model loading
            - Embedding file loading
            - Dataset parsing
            - The backend would run successfully with 4–8 GB memory.
        - **Option B: Deploy on Google Cloud Platform (GCP)**
            - We can deploy using: **Compute Engine (VM)**
            - **Cloud Run** (requires optimized memory settings)
            - **Cloud Functions + Cloud Storage** (if breaking API into microservices)
            - By choosing a VM instance with sufficient RAM, backend deployment will succeed.
    - **Conclusion**
        - **Frontend deployment** → Successfully completed and stable.
        - **Backend deployment** → Attempted but failed due to memory limitations on Render's free tier.
        - **Next step** → Use a paid Render plan or deploy on GCP with increased memory to host the backend successfully

8. References and Appendix

- References drawn from included milestone documents, published papers such as NeuroStylist, Conditional Similarity Networks, and RAGAS documentation.
- Appendix includes sample dataset schemas, scoring sheets, flowcharts, and hyperparameter tuning logs as provided in milestone deliverables.